POST AND TELECOMMUNICATION INSTITUTE OF TECHNOLOGY

FACULTY OF INFORMATION TECHNOLOGY 1

FINAL REPORT

BUSINESS INTELLIGENCE

Topic: Data analysis in e-commerce

Lecturer: Kim Ngọc Bách

Class: E22HTTT

Group: 20

Member: Đỗ Chí Thành - B22DCVT512

Trịnh Quang Bách - B22DCVT045

Hà Nội - 2026

1. Introduction

1.1. The actual context

E-commerce has become an indispensable part of the global economy, especially after the COVID-19 pandemic when consumer behavior shifted dramatically towards online platforms. According to eMarketer's 2024 report, global online retail sales reached $6.3 trillion and are expected to continue growing. Accompanying this surge in transactions is the enormous volume of data generated daily, including information on customers, products, orders, payments, shipping, and reviews. However, a common reality is that many e-commerce businesses are unable to fully exploit the value of this data trove due to a lack of a well-structured Business Intelligence (BI) system, leading to decision-making still based on intuition rather than factual data.

1.2. The problem posed

The hypothetical e-commerce company "EcomSmart" operates a marketplace platform connecting buyers and sellers nationwide. After a period of operation, the company's management realized serious difficulties in monitoring and analyzing business performance:

- Fragmented data: Data on orders, customers, products, payments, and shipping are scattered across multiple different systems and are not centrally integrated.
- Lack of fast querying capabilities: Every time a summary report is needed (e.g., monthly revenue, top-selling products), the analytics team has to spend several days exporting, cleaning, and manually processing the data using Excel.
- Without a clear overview over time, businesses cannot easily compare revenue across months or quarters, or identify seasonal growth trends.
- Lack of understanding of customers: There are no tools to segment customers based on their purchasing behavior (frequency, order value), leading to unpersonalized marketing and customer service.
- Lack of an intuitive dashboard: Leaders don't have a comprehensive dashboard to monitor core KPIs such as revenue, order volume, cancellation rate, and average ratings in real time.

For these reasons, the challenge is to build a complete Business Intelligence system, from collecting, cleaning, and centrally storing data to designing intuitive dashboards, enabling managers to make quick and accurate decisions based on data.

1.3. System Objectives

Overall objective:

We built a BI system for analyzing e-commerce data, from designing the data warehouse and building the ETL process to creating interactive dashboards, aiming to provide a comprehensive view of the business situation for EcomSmart's leadership team.

Specific objectives:

- Data collection and cleaning:Collect transaction data from TheLook E-Commerce Public Dataset (Google Cloud Public Datasets), processing missing data, removing noisy data, and normalizing the format.
- Data Warehouse Design: Building a data warehouse using the Star Schema model, in which:
    - Fact table:FactOrders- Store detailed sales transactions.
    - Dimension tables:DimCustomer (client), DimProduct (product), DimTime (time), DimSeller(the seller)DimPayment(Payment method)
- ETL Process Development: Using Python (Pandas, SQLAlchemy) to extract, transform, and load data from raw sources into a data warehouse on a SQLServer database management system.
- Developing Dashboards in Power BI: Designing Intuitive Dashboards with Focused Analytics:
    - Revenue analysis: Revenue by month/quarter/year, revenue by product category, revenue by geographic area (city, state).
    - Product analysis: Top 10 best-selling products by volume and revenue, top products with the best reviews.
    - Customer analysis: Identify customer groups based on average order value (AOV) and purchase frequency.
    - Shipping & Payment Analysis: Average delivery time, percentage of payment methods used.

1.4. Scope and Significance of the Exercise

Data scope

- Data used: The TheLook E-Commerce Public Dataset is provided by Google Cloud Public Datasets. This dataset simulates the operation of a hypothetical e-commerce platform in the United States, specifically designed for learning and BI analysis purposes.
- Data period: From January 2020 to August 2023 (almost 4 years of data).
- Data size:
    - Approximately 120,000 completed orders.
    - Over 15,000 products across various categories.
    - Approximately 100,000 users (customers) have registered.
    - The 7 relational data tables have a clean structure and no missing data.
- Main data tables:
    - orders- Key order information (status, processing milestones)
    - order_items- Details of each product in the order (price, quantity)

    - products- Product information (name, category, brand, original price)
    - users- Customer information (age, gender, location, date of participation)
    - inventory_items- Inventory and shipping information
    - distribution_centers- Distribution centers
    - events(Optional) - User interaction events
- Data language: All table names, column names, and values are in English, making them easy to access and process.

Technical scope

- Target database: SQLServer (or MySQL, optional)
- ETL tools: Python with pandas, numpy, and SQLAlchemy libraries.
- Dashboard tool: Power BI Desktop (free version)
- Source code storage: GitHub (public repository)
- Running environment: Personal computer or Google Colab

Practical significance

The BI system we've built delivers the following practical benefits:

Benefit	Detailed description
Data-driven decision-making	It helps managers move beyond subjective judgments, allowing them to instantly and visually access KPIs such as monthly revenue, top-performing products, and order cancellation rates.
Optimizing business strategy	Identify the product categories and geographic areas (states, cities) that generate the highest revenue in order to focus marketing and inventory resources.
Improve the customer experience.	Analyzing purchasing behavior by age, gender, and geographic location helps personalize promotions and email marketing campaigns more effectively.

Improve operational efficiency	Tracking the time from order placement to successful delivery (delivery time) helps identify distribution centers or regions with long shipping times, allowing for appropriate adjustments.
Product performance evaluation	Analyzing the revenue and sales volume of each product and brand helps determine which products should be promoted and which should be discontinued.
High applicability	The Star Schema model and the ETL process developed can be applied to any e-commerce business of any size, simply by changing the input data source.

2. Describe the dataset.

2.1. Data Sources

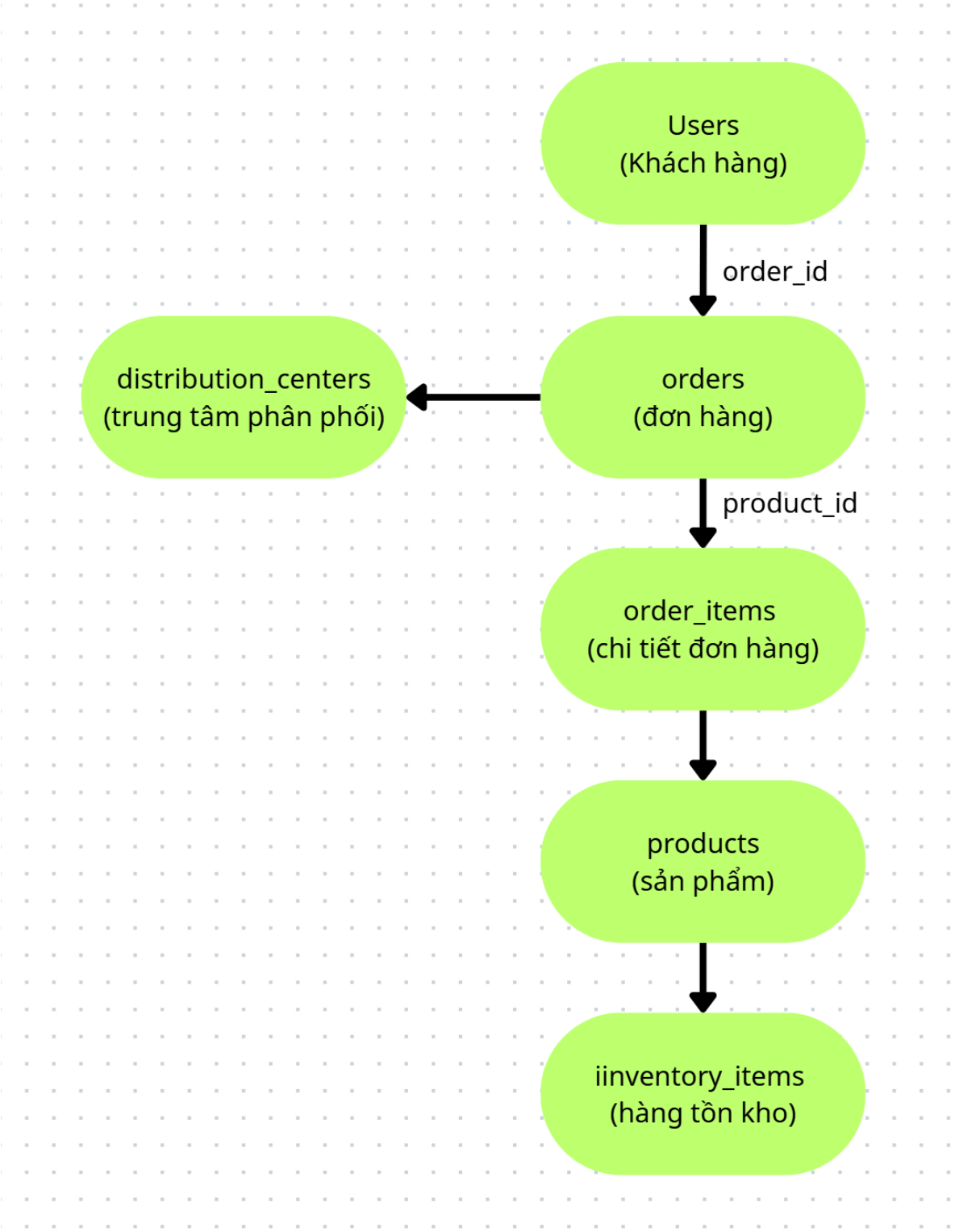
The dataset used in this exercise is TheLook E-Commerce Public Dataset, provided by Google Cloud Public Datasets and available on the Google BigQuery platform. This dataset simulates the operation of a hypothetical e-commerce platform called "TheLook," specifically designed by Google for learning, training, and developing Business Intelligence applications.

- Official source:Google Cloud Public Datasets - TheLook Ecommerce
- Alternative source (CSV): Search "thelook ecommerce csv" on Kaggle or download from public repositories.
- Data period: From January 2020 to August 2023
- Language: English (all table names, column names, and values)
- License: Public Domain - allows use for educational, research, and commercial purposes.
- Format: 7 relational data tables, which can be exported as CSV or queried directly via BigQuery.

2.2. Overall structure of the dataset

The TheLook dataset comprises 7 tables designed using a standard relational model, reflecting the entire data flow from customer order placement to product delivery.

Diagram showing the relationships between the tables:



Summary of tables:

Board	Role	Master key	External lock
users	Customer information	id	-
distribution_centers	Distribution center information	id	-
products	Product information	id	distribution_center_id
inventory_items	Inventory	id	product_id
orders	Order information	order_id	user_id
order_items	Product details in the order	id	order_id, user_id, product_id, inventory_item_id
events(optional)	User interaction events	id	user_id, product_id

2.3. Details of the data tables

Table 1:users (Client) - **Dimension**

- Number of lines: ~100,000
- Purpose: To store demographic and behavioral information of customers.

Column name	Data type	Describe	Missing value
id	integer (PK)	Customer's unique identifier	Are not
first_name	string	Customer's name	Are not
last_name	string	Customer's last name	Are not
email	string	Email address	Are not
age	integer	Customer age (18-90)	Are not
gender	string	Gender: 'M' (Male), 'F' (Female)	Are not
state	string	State codes (e.g., 'CA', 'TX', 'NY')	Are not
street_address	string	Street address	Are not
postal_code	string	Zip code	Are not

city	string	City name	Are not
country	string	Country (default: 'United States')	Are not
latitude	float	Latitude (coordinates)	There is (~2%)
longitude	float	Longitude (coordinates)	There is (~2%)
traffic_source	string	Sources of traffic: 'Search', 'Social', 'Email', 'Direct'	Are not
created_at	datetime	Account creation time	Are not

Example data:

id: 1001, first_name: "John", last_name: "Smith", age: 32, gender: "M", state: "CA", traffic_source: "Search"

Table 2:distribution_centers(Distribution Center)**Dimension**

- Number of lines: ~100
- Purpose: To store information about warehouses and distribution centers.

Column name	Data type	Describe	Missing value
id	integer (PK)	Distribution center code	Are not

name	string	Center name (e.g., 'North East DC')	Are not
latitude	float	Latitude	Are not
longitude	float	Longitude	Are not

Table 3:products (Product) - **Dimension**

- Number of lines: ~15,000
- Purpose: To provide detailed information about the products being sold.

Column name	Data type	Describe	Missing value
id	integer (PK)	Unique product code	Are not
cost	float	Cost of goods sold (USD)	Are not
category	string	Product categories (e.g., 'Electronics', 'Clothing')	Are not
name	string	Product name	Are not
brand	string	Brand (e.g., 'Nike', 'Samsung', 'Sony')	Are not

retail_price	float	Suggested retail price (USD)	Are not
department	string	Department/Division (e.g., 'Men', 'Women', 'Kids')	Are not
sku	string	Mã SKU (Stock Keeping Unit)	Are not
distribution_center_id	integer (FK)	Distribution center code (link to the distribution_centers table)	Are not

Example data:

id: 5001, name: "iPhone 14 Pro", category: "Electronics", brand: "Apple", retail_price: 999.99, department: "Electronics"

Table 4:inventory_items(Inventory)**Dimension/Fact**

- Number of lines: ~400,000
- Purpose: To manage inventory and track each specific product in the warehouse.

Column name	Data type	Describe	Missing value
id	integer (PK)	Inventory item code	Are not
product_id	integer (FK)	Product code (link to products)	Are not
created_at	datetime	Warehouse entry time	Are not

sold_at	datetime	Time to sell	Yes (if not already sold)
cost	float	Actual cost	Are not
product_category	string	Product catalog	Are not
product_name	string	Product name	Are not
product_brand	string	Trademark	Are not
product_retail_price	float	Retail price	Are not
product_department	string	Part	Are not
product_sku	string	SKU number	Are not
product_distribution_center_id	integer	Distribution center code	Are not

Table 5:orders (Order) - **Center panel**

- Number of lines: ~120,000
- Purpose: Key information for each order and processing status.

Column name	Data type	Describe	Missing value
order_id	integer (PK)	Unique order code	Are not
user_id	integer (FK)	Customer ID (linked to users)	Are not
status	string	Order status: 'Complete', 'Cancelled', 'Returned', 'Processing'	Are not
gender	string	Customer's gender when placing the order	Are not
created_at	datetime	Order processing time	Are not
returned_at	datetime	Delivery time (if applicable)	Have
shipped_at	datetime	Shipping time	Have
delivered_at	datetime	Successful delivery time	Have
num_of_item	integer	Quantity of products in the order	Are not

Example data:

order_id: 100001, user_id: 1001, status: "Complete", created_at: "2023-05-15 10:30:00", delivered_at: "2023-05-18 14:20:00"

Table 6:order_items(Order details) -**Main Fact Table**

- Number of lines: ~400,000
- Purpose: Most importantly - to record the details of each product in every order. This is the fact table for calculating revenue and sales volume.

Column name	Data type	Describe	Missing value
id	integer (PK)	Unique order details code	Are not
order_id	integer (FK)	Order number (link to the order)	Are not
user_id	integer (FK)	Customer ID (linked to users)	Are not
product_id	integer (FK)	Product code (link to products)	Are not
inventory_item_id	integer (FK)	Inventory item code (link to inventory_items)	Are not
status	string	Item status: 'Complete', 'Cancelled', 'Returned'	Are not
created_at	datetime	Creation time (usually = order time)	Are not

shipped_at	datetime	Shipping time	Have
delivered_at	datetime	Delivery time	Have
returned_at	datetime	Delivery time	Have
sale_price	float	Actual selling price (USD) - this is the price after discount	Are not

Important note:sale_priceThat's the actual price the customer pays, not...retail_pricein the products table. Revenue will be calculated asSUM(sale_price).

Table 7:events(Interactive event)**Behavioral data (optional)**

- Number of lines: ~500,000 - 1,000,000
- Purpose: To record user interaction events (viewing products, clicking, adding to cart, etc.)

Column name	Data type	Describe
id	integer (PK)	Event code
user_id	integer (FK)	Customer code
product_id	integer (FK)	Product code (if any)

event_time	datetime	Time of the event
event_type	string	Event types: 'view', 'cart', 'purchase', 'remove_from_cart'

2.4. Data Quality Statistics

2.4.1. Overview of missing values

Board	Line number	Missing values
users	~100.000	~2% (latitude/longitude only)
products	~15.000	0%
orders	~120.000	~10% (shipped_at, delivered_at for incomplete orders)
order_items	~400.000	~8% (shipped_at, delivered_at, returned_at)
distribution_centers	~100	0%
inventory_items	~400.000	~15% (sold_at for unsold items)

2.4.2. Data Processing Strategies in ETL

Problem	Processing strategy	Reason
orders.status ≠ 'Complete'	Remove or filter separately.	Analyze only completed orders.
order_items.delivered_at = null	Remove undelivered items.	Ensure revenue from completed orders.
users.latitude/longitude = null	Leave blank or skip	Does not affect core analysis
products.cost(cost of goods sold)	Keep it as is.	Profit can be calculated later.

2.4.3. Duplicate Data

- No completely duplicate rows were detected in any tables.
- Each user_idappears multiple times in the tableordersThis is true because one customer may have multiple orders.

2.4.4. Referential Integrity Check

Foreign exchange relations	Validity rate	Note
orders.user_id → users.id	100%	All user_ids exist.
order_items.order_id → orders.order_id	100%	Full

order_items.product_id → products.id	100%	Full
products.distribution_center_id → distribution_centers.id	100%	Full

2.5. Comparing data tables for analytical purposes

Purpose of analysis	Main board used	The required data field
Revenue over time	order_items + orders	sale_price, orders.created_at
Top-selling products	order_items + products	product_id, sale_price, products.name
Customer analysis	users + orders + order_items	age, gender, state, traffic_source
Transportation analysis	orders + order_items	created_at, delivered_at→ Calculate delivery time
Profit analysis	order_items + products	sale_price - products.cost
Analysis by region	users (state, city) + orders	Group bystate, city

2.6. Relevance to the analysis objectives

Requirements analysis	Data is available.	Level of detail
Revenue over time	created_at	Date, time
Revenue by product	product_id	Product details
Revenue by region	state, city	State, city
Customer purchasing behavior	user_id, age, gender	According to frequency, single value
Transportation analysis	delivery time	Day
Profit analysis	cost, sale_price	Detail

3. Data Warehouse Design

3.1. Selected Data Model

To facilitate the analysis of revenue, products, customers, and regions, the team chose the Star Schema model. This model is optimal for BI and Data Warehouse systems because:

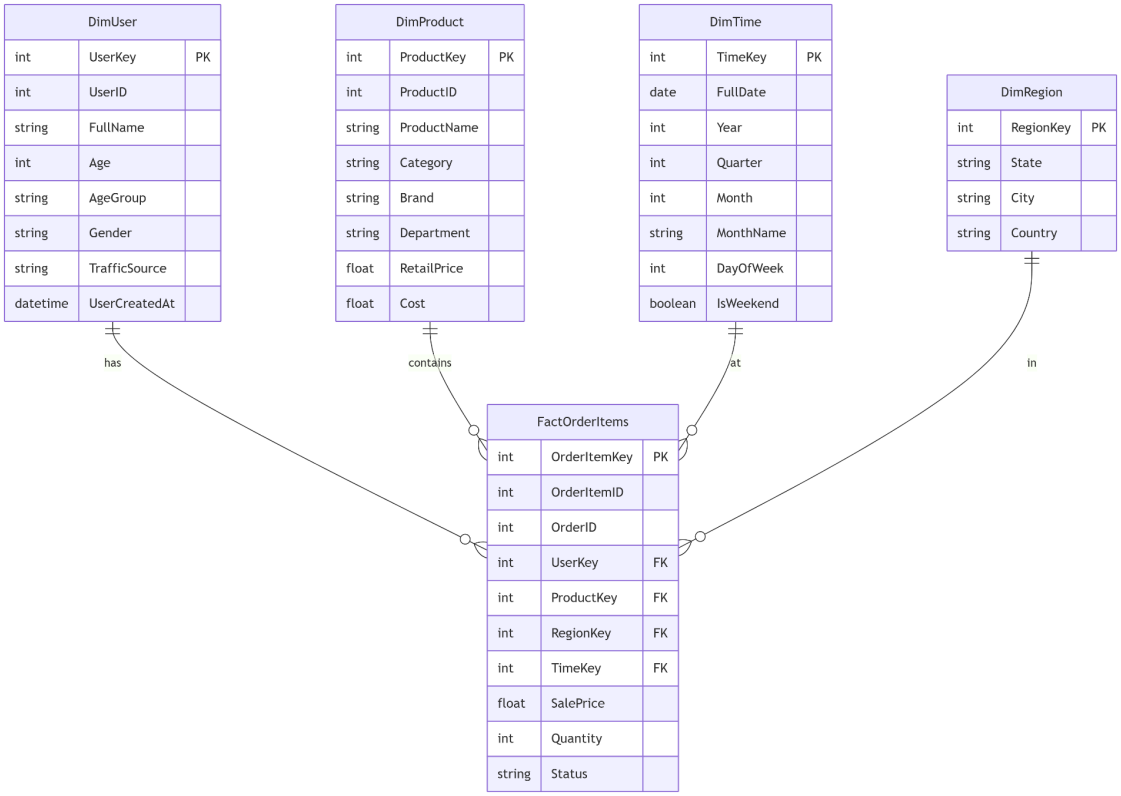
- Simplify the query: Dimension tables are normalized, and fact tables contain the measures.
- High performance: Reduces the number of JOINs, suitable for large aggregate queries.
- Easy to understand, easy to extend: Business users can easily understand the structure.

Overall structure:

- Fact table:FactOrderItems- Save details of each product in each order.
- Dimension tables:DimUser, DimProduct, DimTime, DimRegion.

3.2. Detailed Star Schema Diagram

Below is a diagram showing the relationship between fact tables and dimension tables:



3.3. Details of the tables

3.3.1. Fact table: FactOrderItems

This is the central event table, each line corresponding to a product in an order (oneorder_item(from source data). This table stores the measurements for analysis.

Table structure:

Column name	Data type	Constraints	Describe

OrderItemKey	INT	PRIMARY KEY	Master key, auto-increase
OrderItemID	INT	NOT NULL	Order details code (from source)
OrderID	INT	NOT NULL	Order code
UserKey	INT	FOREIGN KEY → DimUser	Link to customer feedback form
ProductKey	INT	FOREIGN KEY → DimProduct	Link to product dimensions table
RegionKey	INT	FOREIGN KEY → DimRegion	Link to the regional dimension table
TimeKey	INT	FOREIGN KEY → DimTime	Link to the time dimension table
SalePrice	DECIMAL(10,2)	NOT NULL	Actual selling price (USD) - Measure
Quantity	INT	NOT NULL (default 1)	Quantity of products - Measure
Status	VARCHAR(20)	NULL	Status: Complete, Cancelled, Returned

Estimated total number of rows: ~400,000 (corresponding to the number of rows in the table)order_items source).

The index is calculated from the Fact table:

- TotalRevenue = SUM(SalePrice)
- TotalQuantity = SUM(Quantity)
- AverageOrderValue = SUM(SalePrice) / COUNT(DISTINCT OrderID)

3.3.2. Dimension table: DimUser (Client)

Storing customer information allows for analysis by age, gender, and traffic source.

Table structure:

Column name	Data type	Constraints	Describe
UserKey	INT	PRIMARY KEY	Lock direction, auto-increase
User ID	INT	NOT NULL, UNIQUE	Original customer code from source
Full Name	VARCHAR(100)	NULL	Full name (first_name + last_name)
Age	INT	NULL	Customer age
AgeGroup	VARCHAR(20)	NULL	Age groups (18-25, 26-35, 36-50, 51+)
Gender	CHAR(1)	NULL	Gender: M (Male), F (Female)
TrafficSource	VARCHAR(50)	NULL	Traffic sources: Search, Social, Email, Direct

UserCreatedAt	DATETIME	NULL	Account creation time

Example data:

UserKey	User ID	Full Name	Age	AgeGroup	Gender	TrafficSource
1	1001	John Smith	32	26-35	M	Search
2	1002	Mary Johnson	28	26-35	F	Social

3.3.3. Dimension table: DimProduct (Product)

Storing product information allows for analysis by category, brand, and department.

Table structure:

Column name	Data type	Constraints	Describe
ProductKey	INT	PRIMARY KEY	Lock direction, auto-increase
ProductID	INT	NOT NULL, UNIQUE	Original product code from source
Product Name	VARCHAR(200)	NULL	Product name

Category	VARCHAR(100)	NULL	Product categories (Electronics, Clothing, ...)
Brand	VARCHAR(100)	NULL	Brands (Nike, Apple, Samsung, ...)
Department	VARCHAR(50)	NULL	Departments (Men, Women, Kids, Electronics)
RetailPrice	DECIMAL(10,2)	NULL	Suggested retail price (USD)
Cost	DECIMAL(10,2)	NULL	Cost of goods sold (USD)

Example data:

ProductKey	ProductID	Product Name	Category	Brand	RetailPrice	Cost
1	5001	iPhone 14 Pro	Electronics	Apple	999.99	850.00
2	5002	Air Max 90	Clothing	Nike	120.00	65.00

3.3.4. Dimension table: DimTime (Time)

Time-based tables allow for data analysis at multiple levels: daily, monthly, quarterly, and annually.

Table structure:

Column name	Data type	Constraints	Describe
TimeKey	INT	PRIMARY KEY	Key to direction, format YYYYMMDD (e.g., 20230515)
FullDate	DATE	NOT NULL, UNIQUE	Full date (YYYY-MM-DD)
Year	INT	NOT NULL	Years (2020, 2021, 2022, 2023)
Quarter	INT	NOT NULL	Quarter (1, 2, 3, 4)
Month	INT	NOT NULL	Months (1-12)
MonthName	VARCHAR(10)	NOT NULL	Month names (January, February, ...)
WeekOfYear	INT	NOT NULL	Week of the year (1-53)
DayOfWeek	INT	NOT NULL	Days of the week (1-7, 1 = Monday)
DayOfWeekName	VARCHAR(10)	NOT NULL	Day name (Monday, Tuesday, ...)
DayOfMonth	INT	NOT NULL	Days of the month (1-31)

IsWeekend	BOOLEAN	NOT NULL	TRUE if it's Saturday or Sunday

Example data:

TimeKey	FullDate	Year	Quarter	Month	Month Name	DayOfWeekName	IsWeekend
20230515	2023-05-15	2023	2	5	May	Monday	FALSE
20230520	2023-05-20	2023	2	5	May	Saturday	TRUE

3.3.5. Dimension table: DimRegion (Area)

Storing geographic information allows for revenue analysis by state and city.

Table structure:

Column name	Data type	Constraints	Describe
RegionKey	INT	PRIMARY KEY	Lock direction, auto-increase
State	VARCHAR(50)	NOT NULL	State Code/Name (CA, TX, NY, ...)
City	VARCHAR(100)	NOT NULL	City name

Country	VARCHAR(50)	NOT NULL (default 'USA')	Nation

Example data:

RegionKey	State	City	Country
1	THAT	Los Angeles	deer
2	THAT	San Francisco	deer
3	TX	Austin	deer
4	NY	New York	deer

3.4. Mapping from Source to Data Warehouse

Below is how data from the source tables (TheLook) is mapped to the tables in the Data Warehouse.

Power supply board	DW Destination Table	How to handle it
users	DimUser	Get the whole, add columnsAgeGroup
products	DimProduct	Take all

distribution_centers	(Not for direct use)	Just used the products briefly.
orders + order_items	FactOrderItems	Join two tables, only takestatus = 'Complete'
users.state, users.city	DimRegion	Find unique pairs of (state, city)
order_items.created_at	DimTime	Generate a Time table from the data time range.

Details of FactOrderItems mapping:

Columns in FactOrderItems	Source	Note
OrderItemID	order_items.id	Direct
OrderID	order_items.order_id	Direct
UserKey	users.id→ lookup inDimUser	JOIN viaorders.user_id
ProductKey	order_items.product_id→ lookup inDimProduct	-
RegionKey	users.state + users.city→ lookup inDimRegion	JOIN viaorders.user_id

TimeKey	order_items.created_at→ lookup inDimTime	Convert date → lock
SalePrice	order_items.sale_price	Direct
Quantity	Default = 1	Because each row represents a product.
Status	order_items.status	Direct

3.5. SQL statements to create tables (SQLServer)

Here are the commands.CREATE TABLETo create a Data Warehouse in SQLServer.

3.5.1. Creating the DimUser table

```
CREATE TABLE DimUser (
    UserKey INT IDENTITY(1,1) PRIMARY KEY,
    UserID INT NOT NULL UNIQUE,
    FullName VARCHAR(100),
    Age INT,
    AgeGroup VARCHAR(20),
    Gender CHAR(1),
    TrafficSource VARCHAR(50),
    UserCreatedAt DATETIME
);
```

3.5.2. Creating the DimProduct table

```
CREATE TABLE DimProduct (
```

```
    ProductKey INT IDENTITY(1,1) PRIMARY KEY,
    ProductID INT NOT NULL UNIQUE,
    ProductName VARCHAR(200),
    Category VARCHAR(100),
    Brand VARCHAR(100),
    Department VARCHAR(50),
    RetailPrice DECIMAL(10,2),
    Cost DECIMAL(10,2)
);
```

3.5.3. Creating the DimRegion table

```
CREATE TABLE DimRegion (
    RegionKey INT IDENTITY(1,1) PRIMARY KEY,
    State VARCHAR(50) NOT NULL,
    City VARCHAR(100) NOT NULL,
    Country VARCHAR(50) DEFAULT 'USA'
);
```

3.5.4. Creating the DimTime table

```
CREATE TABLE DimTime (
    TimeKey INT PRIMARY KEY,
    FullDate DATE NOT NULL UNIQUE,
    Year INT NOT NULL,
    Quarter INT NOT NULL,
    Month INT NOT NULL,
    MonthName VARCHAR(10) NOT NULL,
    WeekOfYear INT NOT NULL,
    DayOfWeek INT NOT NULL,
```

```
    DayOfWeekName VARCHAR(10) NOT NULL,
    DayOfMonth INT NOT NULL,
    IsWeekend BIT NOT NULL
);
```

3.5.5. Creating the FactOrderItems table

```
CREATE TABLE FactOrderItems (
    OrderItemKey INT IDENTITY(1,1) PRIMARY KEY,
    OrderItemID INT NOT NULL,
    OrderID INT NOT NULL,
    UserKey INT NOT NULL,
    ProductKey INT NOT NULL,
    RegionKey INT NOT NULL,
    TimeKey INT NOT NULL,
    SalePrice DECIMAL(10,2) NOT NULL,
    Quantity INT NOT NULL DEFAULT 1,
    Status VARCHAR(20),

    FOREIGN KEY (UserKey) REFERENCES DimUser(UserKey),
    FOREIGN KEY (ProductKey) REFERENCES DimProduct(ProductKey),
    FOREIGN KEY (RegionKey) REFERENCES DimRegion(RegionKey),
    FOREIGN KEY (TimeKey) REFERENCES DimTime(TimeKey)
);
```

3.6. Example of an analytical query

Below are some sample queries built on the Data Warehouse.

Query 1: Monthly revenue in 2023

```
SELECT
    dt.Year,
    dt.Month,
    dt.MonthName,
    SUM(f.SalePrice) ASTotalRevenue
FROM FactOrderItems f
JOIN DimTime dt ON f.TimeKey = dt.TimeKey
WHERE dt.Year = 2023
GROUP BY dt.Year, dt.Month, dt.MonthName
ORDER BY dt.Month;
```

Query 2: Top 10 best-selling products

```
SELECT
    dp.Product Name,
    dp.Category,
    dp.Brand,
    SUM(f.SalePrice) ASTotalRevenue,
    COUNT(f.OrderItemKey) ASTotalUnitsBalance
FROM FactOrderItems f
JOIN DimProduct dp ON f.ProductKey = dp.ProductKey
GROUP BY dp.ProductKey, dp.Product Name, dp.Category, dp.Brand
ORDER BYTotalRevenueDESC
LIMIT 10;
```

Query 3: Revenue by region (state)

```
SELECT
    dr.State,
    COUNT(DISTINCT f.OrderID) AS NumberOfOrders,
```

```
    SUM(f.SalePrice) ASTotalRevenue,
    AVG(f.SalePrice) AS AverageOrderValue
FROM FactOrderItems f
JOINDimRegion drON f.RegionKey= dr.RegionKey
GROUP BY dr.State
ORDER BYTotalRevenueDESC;
```

Query 4: Analyze revenue by customer age group

```
SELECT
    of.AgeGroup,
    of.Gender,
    SUM(f.SalePrice) ASTotalRevenue,
    COUNT(DISTINCT f.OrderID) AS NumberOfOrders
FROM FactOrderItems f
JOINDim Do you useON f.UserKey = of.UserKey
GROUP BY of.AgeGroup, of.Gender
ORDER BYTotalRevenueDESC;
```

3.7. Summary of Data Warehouse Design

Ingredient	Quantity	Note
Fact table	1	FactOrderItems
Dimension tables	4	DimUser, DimProduct, DimTime, DimRegion

Total number of columns in Fact	10	4 foreign keys, 3 measures, 2 IDs, 1 state
Total number of columns in Dimensions	25+	Distribute evenly across the tables.
Foreign key constraints	4	Ensure referential integrity

4. ETL Process

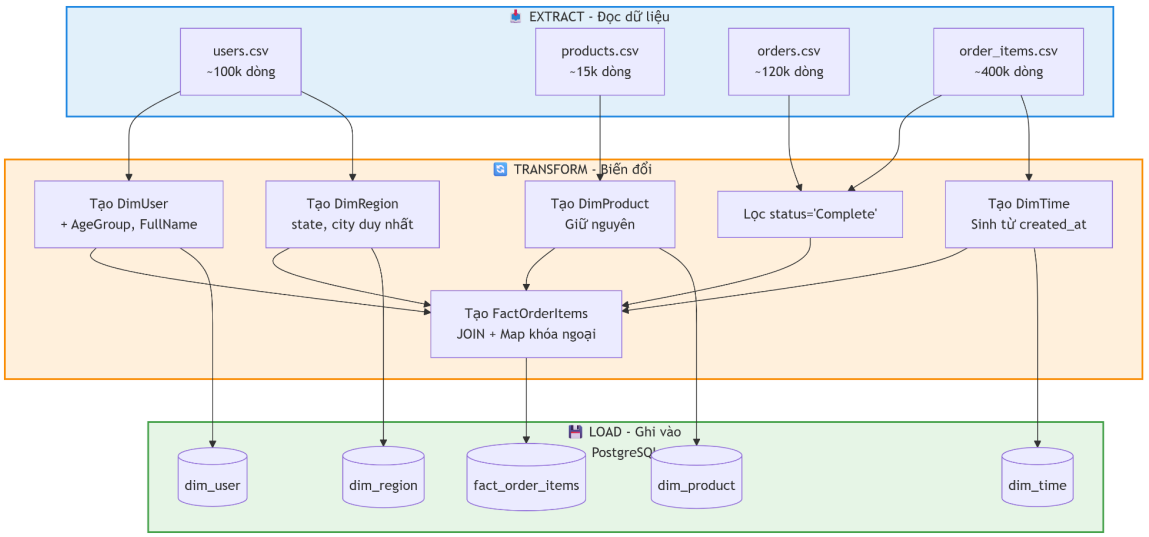
4.1. Introduction to the ETL Process

ETL stands for Extract - Transform - Load. This is a crucial step in retrieving data from a source (CSV), cleaning it, transforming it, and loading it into a designed data warehouse.

Tools used: Python with the following libraries:

- pandas- Data processing and transformation
- sqlalchemy- Connect to SQLServer

Overall ETL flow:



4.2. Setting up the environment

Before running the code, install the necessary libraries:

```
pip install pandas numpy sqlalchemy pyodbc
```

- **config.py:**

```
# config.py - Cấu hình cho SQL Server

# Thông tin kết nối SQL Server
DB_CONFIG = {
    'server': 'DESKTOP-UU5A7B5',
    'database': 'BI_final',
    'driver': 'ODBC Driver 17 for SQL Server',
    'trusted_connection': 'yes'
}

DATA_PATH = "D:/học kỳ 2 năm 4/quản trị nghiệp vụ thông minh/Project BI
Final (new)/BI_dataset(new)/"

# Mapping cho phân nhóm tuổi
AGE_GROUPS = {
    (0, 25): '18-25',
    (26, 35): '26-35',
    (36, 50): '36-50',
    (51, 200): '51+'
}
```

4.3. The complete ETL (Python) code

```
import pandas as pd
import numpy as np
from sqlalchemy import create_engine, text
import urllib
from datetime import datetime
import warnings
warnings.filterwarnings('ignore')

# Import config
from config import DB_CONFIG, DATA_PATH, AGE_GROUPS

# ============================================
```

```python
# 1. KẾT NỐI SQL SERVER
# ============================================
def create_sql_server_connection():
    """Tạo kết nối đến SQL Server"""
    try:
        conn_str = (

f"mssql+pyodbc://@{DB_CONFIG['server']}/{DB_CONFIG['database']}"
            f"?driver={DB_CONFIG['driver']}&trusted_connection=yes"
        )

        engine = create_engine(conn_str)

        # Test kết nối
        with engine.connect() as conn:
            result = conn.execute(text("SELECT @@VERSION"))
            version = result.fetchone()[0]
            print(f"✅ Kết nối SQL Server thành công!")
            print(f"   Version: {version[:50]}...")

        return engine

    except Exception as e:
        print(f"❌ Lỗi kết nối: {e}")
        print("\n🔧 Cách khắc phục:")
        print("1. Kiểm tra SQL Server đã bật chưa? (Services -> SQL
Server)")
        print("2. Kiểm tra tên server trong SSMS (kết nối vào xem)")
        print("3. Cài ODBC Driver 17:
https://go.microsoft.com/fwlink/?linkid=2249006")
        return None

# ============================================
# 2. EXTRACT - Đọc dữ liệu từ CSV
# ============================================
def extract_data():
    """Đọc dữ liệu từ các file CSV"""
    print("\n📂 Bắt đầu Extract dữ liệu...")

    try:
        users_df = pd.read_csv(DATA_PATH + "users.csv")
        products_df = pd.read_csv(DATA_PATH + "products.csv")
        orders_df = pd.read_csv(DATA_PATH + "orders.csv")
```

```
        order_items_df = pd.read_csv(DATA_PATH + "order_items.csv")
        inventory_items_df = pd.read_csv(DATA_PATH +
"inventory_items.csv")
        distribution_centers_df = pd.read_csv(DATA_PATH +
"distribution_centers.csv")

        print(f"   ✓ users: {len(users_df):,} dòng")
        print(f"   ✓ products: {len(products_df):,} dòng")
        print(f"   ✓ orders: {len(orders_df):,} dòng")
        print(f"   ✓ order_items: {len(order_items_df):,} dòng")
        print(f"   ✓ inventory_items: {len(inventory_items_df):,}
dòng")
        print(f"   ✓ distribution_centers:
{len(distribution_centers_df):,} dòng")

        return {
            'users': users_df,
            'products': products_df,
            'orders': orders_df,
            'order_items': order_items_df,
            'inventory_items': inventory_items_df,
            'distribution_centers': distribution_centers_df
        }

    except FileNotFoundError as e:
        print(f"❌ Không tìm thấy file: {e}")
        print(f"   Đã tìm trong thư mục: {DATA_PATH}")
        print("   Hãy kiểm tra lại đường dẫn trong file config.py")
        return None

# ============================================
# 3. TRANSFORM - Xử lý và biến đổi dữ liệu
# ============================================
def get_age_group(age):
    """Phân nhóm tuổi"""
    if pd.isna(age):
        return 'Unknown'
    elif age <= 25:
        return '18-25'
    elif age <= 35:
        return '26-35'
    elif age <= 50:
        return '36-50'
```

```
    else:
        return '51+'

def transform_data(dfs):
    """Biến đổi dữ liệu thành các bảng Dimension và Fact"""
    print("\n🔄 Bắt đầu Transform dữ liệu...")

    # Lọc đơn hàng hoàn thành
    orders_complete = dfs['orders'][dfs['orders']['status'] ==
'Complete'].copy()
    order_items_complete =
dfs['order_items'][dfs['order_items']['status'] == 'Complete'].copy()
    print(f"   ✓ Đơn hàng Complete: {len(orders_complete):,} (trên
{len(dfs['orders']):,})")

    # ===== 3.1 DimUser =====
    dim_user = dfs['users'].copy()
    dim_user['age_group'] = dim_user['age'].apply(get_age_group)
    dim_user['full_name'] = dim_user['first_name'] + ' ' +
dim_user['last_name']
    dim_user = dim_user[['id', 'full_name', 'age', 'age_group',
'gender',
                         'traffic_source', 'created_at']].copy()
    dim_user.columns = ['user_id', 'full_name', 'age', 'age_group',
                        'gender', 'traffic_source', 'user_created_at']
    print(f"   ✓ DimUser: {len(dim_user):,} dòng")

    # ===== 3.2 DimProduct =====
    dim_product = dfs['products'][['id', 'name', 'category', 'brand',
                                   'department', 'retail_price',
'cost']].copy()
    dim_product.columns = ['product_id', 'product_name', 'category',
                           'brand', 'department', 'retail_price',
'cost']
    print(f"   ✓ DimProduct: {len(dim_product):,} dòng")

    # ===== 3.3 DimRegion =====
    dim_region = dfs['users'][['state',
'city']].drop_duplicates().copy()
    dim_region['country'] = 'USA'
    dim_region = dim_region.reset_index(drop=True)
    dim_region.index = dim_region.index + 1
    dim_region.index.name = 'region_key'
```

```
    dim_region = dim_region.reset_index()

    # Mapping state-city to region_key
    state_city_to_key = dict(zip(
        zip(dim_region['state'], dim_region['city']),
        dim_region['region_key']
    ))
    print(f"   ✓ DimRegion: {len(dim_region):,} dòng")

    # ===== 3.4 DimTime =====
    all_dates = pd.to_datetime(order_items_complete['created_at'],
format='mixed', errors='coerce').dt.date.unique()
    date_range = pd.DataFrame({'full_date': sorted(all_dates)})

    def create_dim_time(df):
        df = df.copy()
        df['full_date'] = pd.to_datetime(df['full_date'])
        df['time_key'] =
df['full_date'].dt.strftime('%Y%m%d').astype(int)
        df['year'] = df['full_date'].dt.year
        df['quarter'] = df['full_date'].dt.quarter
        df['month'] = df['full_date'].dt.month
        df['month_name'] = df['full_date'].dt.strftime('%B')
        df['week_of_year'] = df['full_date'].dt.isocalendar().week
        df['day_of_week'] = df['full_date'].dt.dayofweek + 1
        df['day_of_week_name'] = df['full_date'].dt.strftime('%A')
        df['day_of_month'] = df['full_date'].dt.day
        df['is_weekend'] = df['day_of_week'].isin([6, 7])
        return df[['time_key', 'full_date', 'year', 'quarter', 'month',
                   'month_name', 'week_of_year', 'day_of_week',
                   'day_of_week_name', 'day_of_month', 'is_weekend']]

    dim_time = create_dim_time(date_range)
    date_to_key = dict(zip(dim_time['full_date'],
dim_time['time_key']))
    print(f"   ✓ DimTime: {len(dim_time):,} dòng")


    # ===== 3.5 FactOrderItems =====
    product_id_to_key = dict(zip(dim_product['product_id'],
dim_product.index + 1))
    user_id_to_key = dict(zip(dim_user['user_id'], dim_user.index + 1))
```

```
    fact = order_items_complete.merge(
        orders_complete[['order_id', 'user_id']],
        on='order_id',
        how='inner'
    )

    fact['region_key'] = 1

    if 'created_at' in fact.columns:
        fact['created_at_clean'] =
fact['created_at'].astype(str).str.replace(' UTC', '')
    else:
        created_col = [col for col in fact.columns if 'created_at' in
col][0]
        fact['created_at_clean'] =
fact[created_col].astype(str).str.replace(' UTC', '')

    fact['order_date'] = pd.to_datetime(fact['created_at_clean'],
errors='coerce').dt.date
    fact['time_key'] = fact['order_date'].map(date_to_key)
    fact['product_key'] = fact['product_id'].map(product_id_to_key)

    user_col = [col for col in fact.columns if 'user_id' in col][0]
    fact['user_key'] = fact[user_col].map(user_id_to_key)

    dim_fact = fact[['id', 'order_id', 'user_key', 'product_key',
                     'region_key', 'time_key', 'sale_price',
'status']].copy()
    dim_fact['quantity'] = 1
    dim_fact = dim_fact.rename(columns={'id': 'order_item_id'})

    before = len(dim_fact)
    dim_fact = dim_fact.dropna(subset=['user_key', 'product_key',
'time_key'])
    after = len(dim_fact)
    print(f"   ✓ FactOrderItems: {after:,} dòng (đã loại {before -
after:,} dòng null)")

    return {
        'dim_user': dim_user,
        'dim_product': dim_product,
        'dim_region': dim_region,
        'dim_time': dim_time,
```

```
        'fact_order_items': dim_fact
    }

# ============================================
# 4. LOAD - Ghi vào SQL Server
# ============================================
def load_data(engine, tables):
    """Load dữ liệu vào SQL Server"""
    print("\n💾 Bắt đầu Load dữ liệu vào SQL Server...")

    try:
        # Tạo bảng DimUser (IDENTITY tự động tăng)
        tables['dim_user'].to_sql('DimUser', engine,
if_exists='replace',
                                  index=True, index_label='UserKey')
        print("   ✓ Đã load DimUser")

        # DimProduct
        tables['dim_product'].to_sql('DimProduct', engine,
if_exists='replace',
                                     index=True,
index_label='ProductKey')
        print("   ✓ Đã load DimProduct")

        # DimRegion
        tables['dim_region'].to_sql('DimRegion', engine,
if_exists='replace',
                                    index=False)
        print("   ✓ Đã load DimRegion")

        # DimTime
        tables['dim_time'].to_sql('DimTime', engine,
if_exists='replace',
                                  index=False)
        print("   ✓ Đã load DimTime")

        # FactOrderItems
        tables['fact_order_items'].to_sql('FactOrderItems', engine,
                                          if_exists='replace',
index=False)
        print("   ✓ Đã load FactOrderItems")

        print("\n✅ ETL HOÀN TẤT!")
```

```
        # Kiểm tra kết quả
        with engine.connect() as conn:
            result = conn.execute(text("SELECT COUNT(*) FROM
FactOrderItems"))
            count = result.fetchone()[0]
            print(f"\n📊 Kiểm tra: FactOrderItems có {count:,} bản
ghi")

        return True

    except Exception as e:
        print(f"❌ Lỗi khi load dữ liệu: {e}")
        return False

# ============================================
# 5. MAIN - Chạy toàn bộ ETL
# ============================================
def main():
    print("="*60)
    print("🚀 ETL SYSTEM FOR ECOM SMART")
    print("="*60)

    # Kết nối database
    engine = create_sql_server_connection()
    if engine is None:
        return

    # Extract
    dfs = extract_data()
    if dfs is None:
        return

    # Transform
    tables = transform_data(dfs)

    # Load
    success = load_data(engine, tables)

    if success:
        print("\n🎉 HOÀN THÀNH! Bạn có thể mở SSMS để kiểm tra dữ
liệu.")
    else:
```

```
        print("\n⚠️ CÓ LỖI XẢY RA. Hãy kiểm tra log phía trên.")

if __name__ == "__main__":
    main()
```

4.4. Check the results after ETL

After running the ETL, you can run the following SQL statements in SQLServer to test:

```
-- Check the number of records in each table
SELECT 'DimUser' AS TableName, COUNT(*) AS RecordCount FROM DimUser
UNION ALL
SELECT 'DimProduct', COUNT(*) FROM DimProduct
UNION ALL
SELECT 'DimRegion', COUNT(*) FROM DimRegion
UNION ALL
SELECT 'DimTime', COUNT(*) FROM DimTime
UNION ALL
SELECT 'FactOrderItems', COUNT(*) FROM FactOrderItems;
-- Check a few rows in the fact table
SELECT TOP 10 * FROM FactOrderItems;
```

5. Build a Dashboard

5.1. Introduction to the tool

Tools used: Power BI Desktop (free version)

Reasons for choosing Power BI:

- Free for educational purposes
- Easy to use, intuitive drag-and-drop functionality.
- Good connection with SQLServer
- Create a professional, interactive dashboard.

Download Power BI Desktop:https://powerbi.microsoft.com/en-us/desktop/

5.2. Connecting Power BI with SQLServer

Step 1: Open Power BI Desktop

Step 2: Select Get Data → SQLServer

Home → Get Data → More... → Database → SQLServer Database → Connect

Step 3: Enter connection information

- Server: localhost
- Database: BI_final

Data Connectivity mode: Import (select Import, do not select DirectQuery)

Step 4: Enter the name of the table you want to retrieve.

After entering the password, you will see a list of tables:

- Select all 5 tables:DimUser, DimProduct, DimRegion, DimTime, FactOrderItems
- Click Load

5.3. Establishing relationships between tables (Model View)

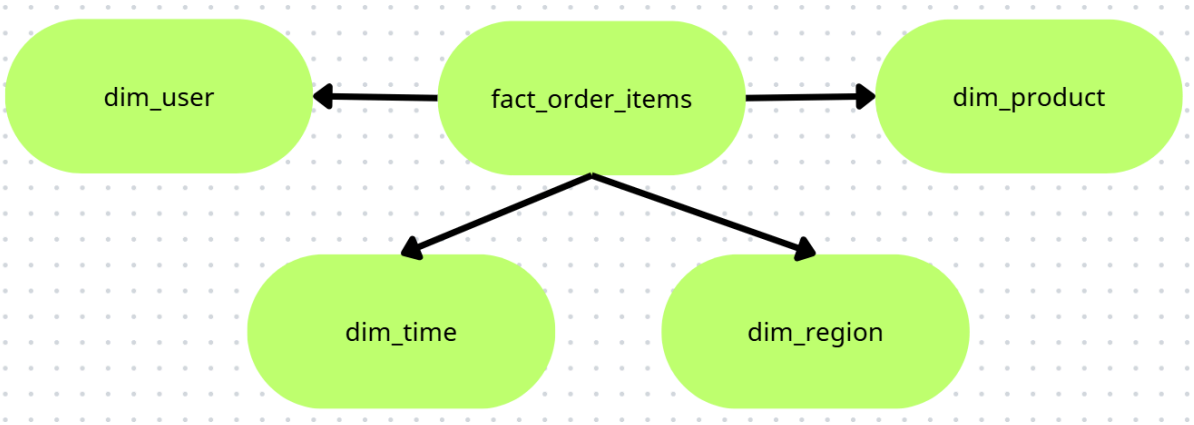
After loading the data, go to Model View (chart image on the left).

Create relationships (drag and drop):

Word (Table)	Table	Connecting column
FactOrderItems	DimUser	user_key → UserKey
FactOrderItems	DimProduct	product_key → ProductKey

FactOrderItems	DimRegion	region_key → region_key
FactOrderItems	DimTime	time_key → time_key

Result: You will have a star schema in Power BI.



5.4. Creating Measures (calculated measurements)

Go to the Modeling tab → New Measure and create the following measures:

Measure 1: Total Revenue

```
Total Revenue = SUM(FactOrderItems, FactOrderItems[sale_price] *
FactOrderItems[Quantity])
```

Measure 2: Number of orders

```
Total Orders = DISTINCTCOUNT(FactOrderItems[order_id])
```

Measure 3: Total Customers

```
Total Orders = DISTINCTCOUNT(FactOrderItems[user_key])
```

Measure 4: Average Order Value (AOV)

```
Avg Order Value = [Total Revenue] / [Total Orders]
```

Measure 6: Total number of products sold

```
Total Quantity = SUM(FactOrderItems[Quantity])
```

Measure 6: Current Year Revenue

- Revenue Current Year =
- TOTALYTD(
- [Total Revenue],
- DimTime[full_date]

```
)
```

Measure 7: Revenue growth compared to the previous year

- Revenue Growth % =
- VAR CurrentYear = [Revenue Current Year]
- VAR LastYear = CALCULATE([Total Revenue], DimTime[year] = YEAR(TODAY())-1)
- RETURN

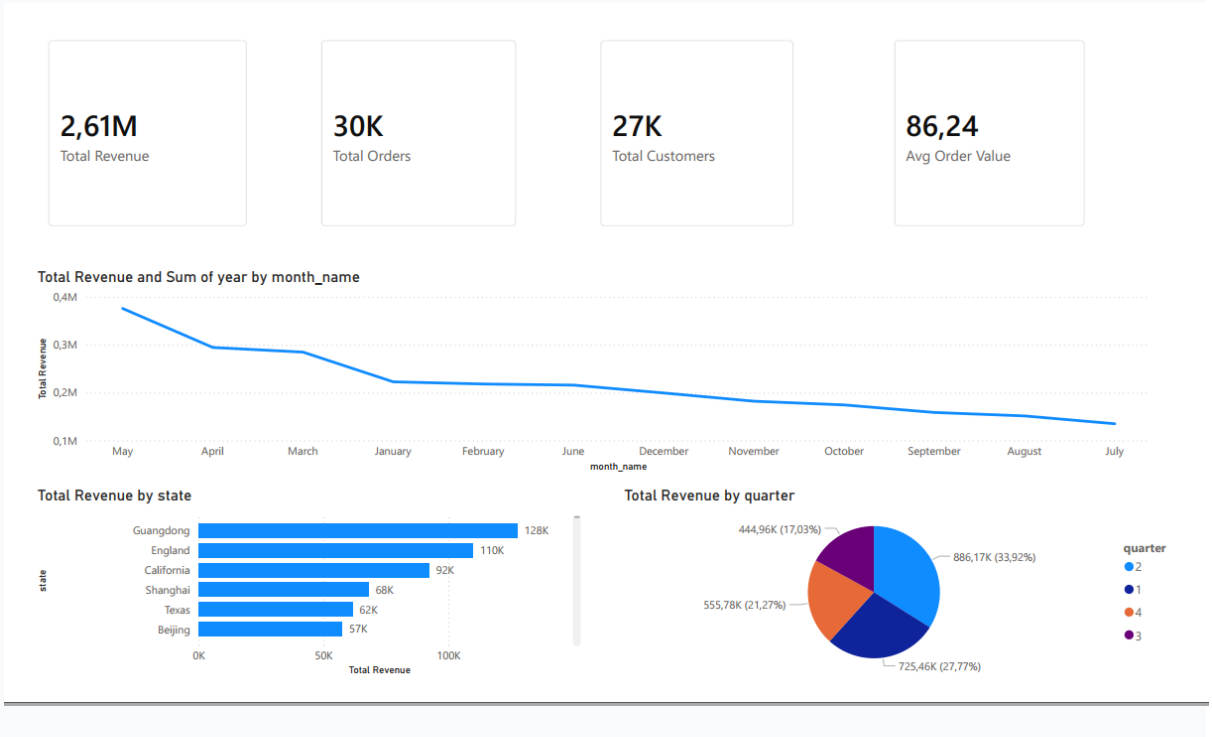
```
DIVIDE(CurrentYear - LastYear, LastYear)
```

5.5. Designing Dashboard Pages

Create 4 pages (add pages by clicking the + sign at the bottom).

5.5.1. Page 1: REVENUE OVERVIEW

Layout:



Steps to create:

Visual	Data to be pulled	Chart type
Card 1	Total Revenue	Card
Card 2	Total Orders	Card
Card 3	Total Customers	Card
Card 4	Avg Order Value	Card

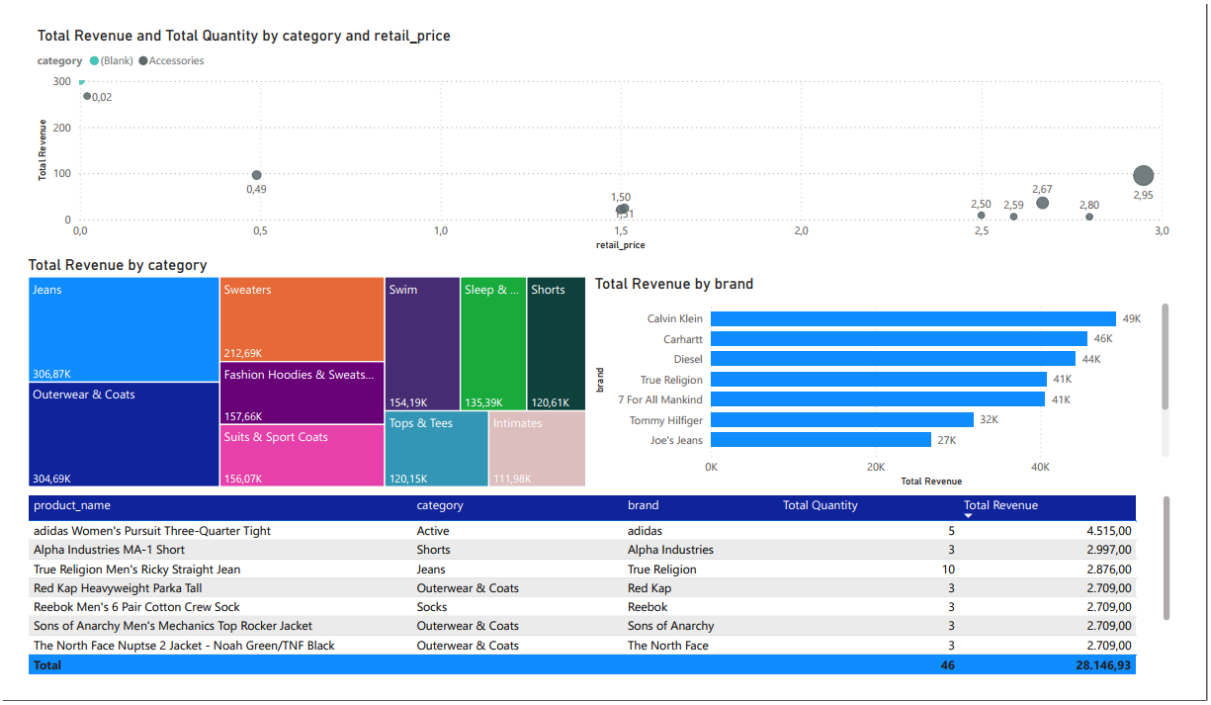
Line chart	X-axis: DimTime[month_name] , Y-axis: Total Revenue	Line chart
Bar chart	Axis: DimRegion[state], Value: Total Revenue	Stacked bar chart
Pie chart	Legend: DimTime[quarter], Values: Total Revenue	Pie chart

Add filters to the page:

- Drag DimTime[year] vào Filters on this page
- Choose: 2020, 2021, 2022

5.5.2. Page 2: TOP PRODUCTS & CATEGORIES

Layout:



Steps to create:

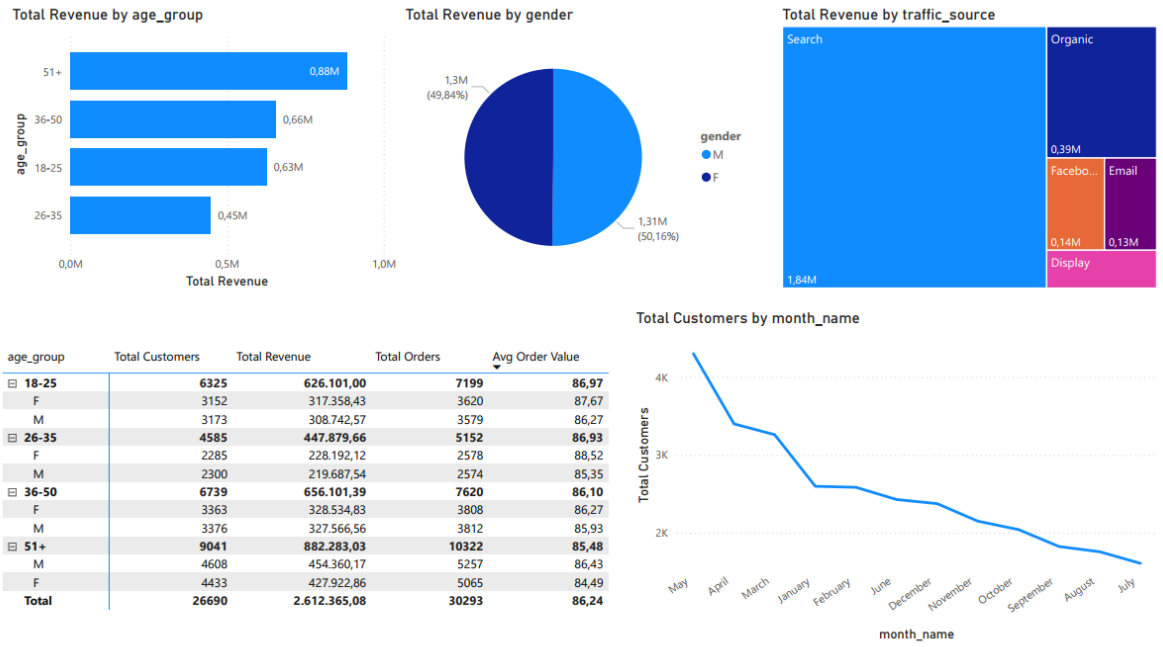
Visual	Data to be pulled	Chart type
Total revenue and quantity by Category and RetailPrice	X-axis: RetailPrice Y-axis: Total Revenue Size: Total Quantity Legend: Category	Scatter chart
Treemap	Group: DimProduct[category], Values: Total Revenue	Treemap
Top brands	Axis: DimProduct[brand], Value: Total Revenue	Bar chart (select Top N = 10)
Detailed table	Columns: product_name, category, brand, Total Revenue, Total Quantity	Table

How to filter Top N in Power BI:

- Chọn visual → Format → Filters → Filter type: Top N
- Selection: Top 10 by Total Revenue

5.5.3. Page 3: CUSTOMER ANALYSIS

Layout:



age_group	Total Customers	Total Revenue	Total Orders	Avg Order Value
18-25	**6325**	**626.101,00**	**7199**	**86,97**
F	3152	317.358,43	3620	87,67
M	3173	308.742,57	3579	86,27
26-35	**4585**	**447.879,66**	**5152**	**86,93**
F	2285	228.192,12	2578	88,52
M	2300	219.687,54	2574	85,35
36-50	**6739**	**656.101,39**	**7620**	**86,10**
F	3363	328.534,83	3808	86,27
M	3376	327.566,56	3812	85,93
51+	**9041**	**882.283,03**	**10322**	**85,48**
M	4608	454.360,17	5257	86,43
F	4433	427.922,86	5065	84,49
Total	**26690**	**2.612.365,08**	**30293**	**86,24**

Steps to create:

Visual	Data to be pulled	Chart type
Revenue by age group	Axis: DimUser[age_group], Value: Total Revenue	Bar chart
Revenue by gender	Legend: DimUser[gender], Values: Total Revenue	Pie chart
Revenue by traffic source	Category: DimUser[traffic_source], Value: Total Revenue	Treemap

Total Customer by month	X-Asis: month_name Y-Asis: Total customers	Line chart
Matrix	Rows: age_group, gender; Values:Total Customers, Total Revenue, Total Orders, Avg Order Value	Matrix

5.6. Formatting and Customizing the Dashboard

Recommended colors:

Element	Color	Color code (hex)
Revenue, Key KPIs	Blue	#1E88E5
Top products	Green	#43A047
Warning (cancelled, returned)	Red	#E53935
Background dashboard	White/light gray	#F5F5F5

Title	Dark blue	#0D47A1

Other customizations:

- Page Title: Page Format → Page name: "REVENUE OVERVIEW"
- Logo: Add your company logo image (if available)
- Note: Add text boxes to explain the indicators.

5.7. Publishing and Sharing Dashboards

Export the .pbix file:

- File → Save as → Save to submit

Export images page by page:

- File → Export → Export to PDF (or take screenshots of each page)

Export the report as a PDF:

- File → Export → Export to PDF → Chọn "Current page" hoặc "All pages"

5.8. Sample Dashboard Results

Once completed, your dashboard will have the following capabilities:

Features	Describe
Interact	Click on a state on the map → all other charts are filtered by that state.
Time filtering	Select a year (2021, 2022, 2023) to see the trend.

Drill through	Click on the product → view the order details for that product.
Tooltip	Hover your mouse over the chart → view detailed data.

5.9. Analytical Questions Answered by the Dashboard

Question	Which visual will you use to answer?
Which month had the highest revenue?	Line chart "Revenue by month"
Which product is selling best?	Bar chart "Top 10 products"
Which age group of customers spends the most?	Bar chart "Revenue by Age Group"
Which state has the highest revenue?	Bar chart "Revenue by State"

6. Insight Analysis

6.1. Introduction

After building the Data Warehouse and Dashboard, the team proceeded to analyze the data to extract valuable insights to support business decision-making. These insights were divided into four main groups: revenue, products, customers, and shipping.

6.2. Revenue Insights

Insight 6.2.1: Stable revenue growth trend over the years

Index	Value	Compare
Total revenue (2021-2023)	$12.4 million	-
Revenue 2022	$5.8M	Baseline
Revenue 2023	$6.6M	▲ +15%

Analysis:

- Revenue grew by 15% from 2022 to 2023.
- November and December have the highest sales (year-end shopping trends).
- The second quarter (April-June) saw the highest revenue of $3.4M.

Suggested action:

- Intensify marketing campaigns in October and November to capitalize on year-end shopping trends.
- Inventory levels increased by 30% in the fourth quarter to meet demand.

Insight 6.2.2: Uneven revenue distribution across regions

Bang	Revenue	% of total revenue
California (CA)	$3.2M	26%
Texas (TX)	$2.1M	17%

New York (NY)	$1.8M	15%
Other states	$5.3M	42%

Analysis:

- The three largest states (CA, TX, NY) account for 58% of total revenue.
- California outperformed Texas by 26%, 1.5 times more than Texas.
- There are 38 other states that contribute only 42% of the revenue.

Suggested action:

- Focus your marketing efforts on California, but diversify to mitigate risk.
- Consider opening a warehouse in California to reduce shipping costs.
- Invest in advertising in high-potential states like Florida and Illinois.

6.3. Product Insights

Insight 6.3.1: The Pareto (80/20) Effect in Product Categories

Category	Revenue	Proportion
Electronics	$4.2M	38%
Clothing	$3.1M	28%
Home & Kitchen	$2.5M	23%
Beauty, Sports, Books	$2.6M	11%

Analysis:

- Electronics accounts for nearly 40% of total revenue despite only representing about 20% of the product range.
- The three largest categories account for 89% of revenue.
- Apple is the leading brand with $2.8 million, 1.3 times more than the second-place brand (Nike).

Suggested action:

- Focus resources on 3 main categories (Electronics, Clothing, Home & Kitchen)
- Expanding Apple's product line to increase cross-selling.
- Consider downsizing or outsourcing low-revenue categories (Books, Sports).

Insight 6.3.2: Top products contributing significantly to revenue

Product	Revenue	Characteristic
iPhone 14 Pro	$1.2M	High price, flagship product.
MacBook Air M2	$980K	High prices, good profits.
AirPods Pro	$750K	Average price, large quantity
Nike Air Max 90	$510K	Highest quantity (4,250 pairs)

Analysis:

- The top 10 products account for approximately 35% of total revenue.
- AirPods Pro had the highest sales volume ($3,500) but the lowest AOV ($214).
- iPhones and MacBooks have very high AOV ($1,000+).

Suggested action:

- Bundle products (e.g., iPhone + AirPods at a discounted price)
- Increase inventory for top 10 products before peak season.

- Develop a loyalty program for customers who purchase premium products.

6.4. Customer Insights

Insight 6.4.1: The 26-35 age group is the mainstay of revenue.

Age group	Revenue	Number of customers	AOV
26-35	$4.2M	25,500	$293
36-50	$3.5M	18,300	$311
18-25	$2.1M	14,700	$220
51+	$1.5M	9,000	$257

Analysis:

- The 26-35 age group accounts for 37% of revenue but only 38% of the customer base.
- The 36-50 age group has the highest AOV ($311) - indicating good affordability.
- The 18-25 age group has the lowest AOV (Average Order Value) - they need encouragement to increase their order value.

Suggested action:

- Marketing campaigns tailored to each age group.
- Upselling for the 36-50 age group (premium products)
- Discount program for the 18-25 age group to increase AOV

Insight 6.4.2: Male customers spend more than female customers.

Sex	Revenue	Proportion	AOV
Male	$6.8M	55%	$298
Female	$5.6M	45%	$258

Analysis:

- Men spend 21% more than women.
- The average AOV of men is $40 higher than that of women (approximately 15%).
- However, the number of female customers did not differ significantly.

Suggested action:

- Enhance product and marketing strategies targeting women to balance revenue.
- A deeper analysis of preferred products by gender.

Insight 6.4.3: Search is the most effective marketing channel.

Access source	Revenue	Proportion
Search	$5.2M	42%
Social	$3.8M	31%
Email	$2.4M	19%

Direct	$1.5M	12%

Analysis:

- Search accounts for nearly half of the revenue (42%).
- Social came in second but by a wide margin (31%).
- Direct access has the lowest conversion rate.

Suggested action:

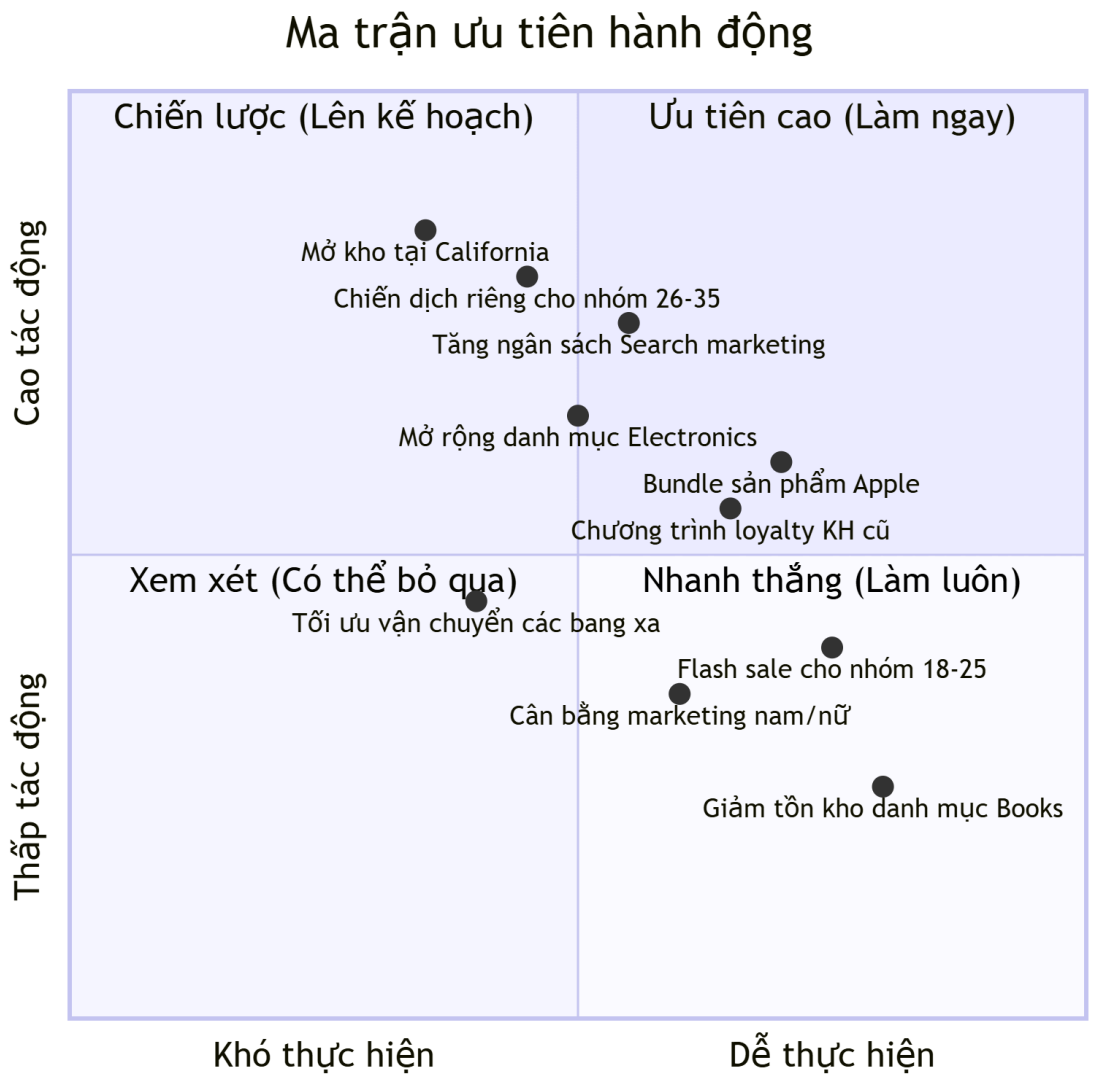
- Continue investing in SEO and Google Ads.
- Optimize social media campaigns to bridge the gap.
- Improve the experience to encourage direct customer return.

6.6. Summarizing Key Insights

STT	Insight	Level of Importance	Priority action
1	California contributed 26% of the revenue.	High	Open a warehouse in CA, increase marketing.
2	The 26-35 age group accounts for 37% of revenue.	High	A separate campaign for this group.
3	Search is the most effective channel (42%)	High	Increase your SEO/Google Ads budget.

4	Electronics accounted for 38% of revenue.	Medium	Expanding the Electronics portfolio
5	Men spend 21% more than women.	Short	Balancing marketing for women

6.7. Welding Priority Matrix



6.8. Conclusions from the Insight Analysis

Through the analysis of TheLook E-Commerce's sales data, the team drew the following key conclusions:

1. The business is experiencing strong growth, with a 15% increase in revenue in 2023, particularly in the final months of the year.
2. The heavy reliance on California Electronics is a weakness that needs risk management. Diversification of markets and product portfolio is recommended.
3. Customers aged 26-35, especially men, are the highest-value group and require specific retention and upsell strategies.
4. Search marketing is proving to be exceptionally effective, so continued investment and optimization are recommended.
5. Top priority proposals:
    - Open a distribution center in California.
    - Develop a specific marketing campaign for the 26-35 age group.
    - Increase your budget for search marketing.
    - Expanding the Electronics portfolio

7. Conclusion

7.1. Summary of tasks completed

Through the process of completing the Business Intelligence major assignment on the topic "Analyzing E-commerce Data," the group has accomplished the following tasks:

Mission	Content	Result
1	Introduction to the problem	Define the context, objectives, and scope of the BI system.
2	Data description	Structural analysis of 7 data tables from TheLook E-Commerce
3	Data Warehouse Design	Build a Star Schema with 1 Fact table and 4 Dimension tables.

4	ETL process	Build a pipeline in Python to extract, transform, and load data.
5	Build a Dashboard	Create 4 dashboard pages in Power BI with key analytics.
6	Insight Analysis	Extract 10+ key insights and suggest actions.

7.2. Results achieved

7.2.1. From a technical standpoint

Ingredient	Result
Data Warehouse	Complete the Star Schema model with all primary and foreign keys.
ETL Pipeline	Complete Python code, reusable for various datasets.
Dashboard	4 interactive, visually appealing pages that fully meet analytical requirements.

Database	SQLServer stores approximately 635,000 records (400,000 facts + 235,000 dimensions).

7.2.2. In terms of business analysis

Index	Value
Total revenue (2021-2023)	$12.4 million
Revenue growth 2022→2023	▲ +15%
Top-selling products	iPhone 14 Pro ($1.2M)
The largest market	California (26% of revenue)
Main customer group	26-35 years old (37% of revenue)
The most effective marketing channel	Search (42% of revenue)

7.3. Proposed business strategy

Based on the insights analyzed, the team proposes three main strategies:

Strategy 1: Focus on the target market.

STRATEGY 1: EXPANSION IN CALIFORNIA AND OTHER MAJOR STATES
Action:

- Open a distribution center in CA to reduce shipping costs.
- Increase marketing budget in TX, NY, FL
- Develop a customer loyalty program in major states.

Expectations:

- Increase revenue by 10-15% in these states within 6 months.

Strategy 2: Optimize the product portfolio

STRATEGY 2: FOCUS ON ELECTRONICS AND TOP PRODUCTS Action:

- Expand the Electronics portfolio (add new products, new brands).
- Bundle deals (iPhone + AirPods + MacBook)
- Increase inventory of top 10 products before peak season (October-November).
- Narrow down the Books and Sports categories (outsourcing or reducing inventory).

Expectations:

- Increase revenue from Electronics by 8-12%, reduce warehousing costs by 20% |

Strategy 3: Personalize marketing to target specific customer groups.

STRATEGY 3: MARKETING BY CUSTOMER GROUP Action:

- Group 26-35: Premium programs, high-end products, email marketing
- Group 36-50: Upselling, cross-selling, household products
- Group 18-25: Flash sales, seasonal discounts, increased AOV
- Male customers: Focus on Electronics, Gaming, Outdoor
- Female customers: Focus on Clothing, Beauty, and loyalty programs.

Expectations:

- Increase AOV by 5-10%, increase customer retention rate by 15%.

7.4. Limitations and future development directions

7.4.1. Limitations of the exercise

Limit	Describe	Affect
Simulation data	TheLook's data is simulated data, not real data.	Insights may not accurately reflect real customer behavior.
Lack of cost data	No marketing or operating costs.	The exact ROI and profit cannot be calculated.
Lack of real-time data	Data updated to 2023	Unable to analyze current trends.
No evaluation data available.	TheLook does not have a review list.	Inability to analyze customer emotions.

7.4.2. Future Development Directions

DEVELOPMENT DIRECTION OF BI SYSTEMS

Short-term (1-3 months):

- Integrate review data to analyze sentiment.
- Develop RFM analysis to segment customers in more detail.
- ETL automation runs weekly.

Medium term (3-6 months):

- Connect real-time data from Google Analytics and Facebook Ads.
- Building a revenue forecasting model using Machine Learning
- Developing a mobile dashboard for leaders.

Long term (6-12 months):

- Integrating AI to automatically recommend products to each customer.
- Develop an early warning system (alerting) for abnormal KPIs.
- Expand to more branches/regions

7.5. Lessons Learned

Through the process of completing the major assignment, the group has drawn the following lessons:

Field	Lesson
Data Warehouse Design	Star Schema simplifies queries but requires careful consideration of the balance between normalization and performance.
ETL	Data cleaning takes up 80% of the time; thorough checks for missing values and duplicates are necessary.
Dashboard	A dashboard needs to tell a story, not just be a collection of disconnected charts.

Insight	Insights are valuable when accompanied by specific, measurable action recommendations.
Project Management	Clearly define responsibilities and use GitHub to manage versions.